Task 1: Create the FoodieAppKey in AWS KMS

Procedure
1. Open the AWS Management Console.
2. Search for Key Management Service (KMS).
3. Choose Customer managed keys → Create key.
4. Select Symmetric as the key type.
5. Select Encrypt and decrypt as the key usage.
6. Enter an alias such as alias/FoodieAppKey.
7. Configure key administrators and key users according to the principle of least privilege.
8. Review the configuration and create the key.
9. Record the key ARN from the key details page.
Example ARN format:
arn:aws:kms:ap-south-1:123456789012:key/example-key-id

Task 2: Enable automatic key rotation

Procedure
1. Open AWS KMS and select alias/FoodieAppKey.
2. Open the key's configuration or key rotation section.
3. Enable automatic key rotation if the key type and configuration support it.
4. Verify the rotation status on the key details page.

Task 3: AWS KMS vs. Azure Key Vault

Feature	AWS KMS	Azure Key Vault
Key management	Create and manage cryptographic keys	Create and manage cryptographic keys
Rotation	Supports automatic rotation for eligible keys	Supports automatic rotation policies for supported keys

Access control	IAM policies and KMS key policies	Azure RBAC or Key Vault access policies, depending on configuration
Integration	Integrates with AWS services such as S3 and EBS	Integrates with Azure services such as Storage and other Azure workloads
Monitoring	AWS CloudTrail and related monitoring tools	Azure Monitor, diagnostic logs and related tools

Two similarities
1. Both services centralize key management and restrict access through permissions.
2. Both support rotation strategies and audit capabilities to improve key security.

Two differences
1. AWS KMS is designed around AWS key-management and service integrations, while Azure Key Vault is integrated with the Azure identity and resource ecosystem.
2. Their permission models and rotation configuration workflows differ. Automatic rotation availability and supported key types also depend on the specific configuration and service features.

Task 4: Generate a data encryption key and encrypt an order

AWS KMS can generate a data encryption key (DEK) using the FoodieAppKey. The plaintext DEK is used locally to encrypt the order, while the encrypted copy of the DEK can be stored alongside the encrypted data.

This is called envelope encryption.

The following Python example uses boto3 and the cryptography library. It generates a DEK through KMS, encrypts a sample order using AES-256-GCM, and then decrypts it.

```
import base64
import boto3
import json
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

kms = boto3.client("kms", region_name="ap-south-1")

# Replace this with your actual KMS key ARN.
key_id =
"arn:aws:kms:ap-south-1:123456789012:key/YOUR-KEY-ID"

# Generate a data encryption key.
result = kms.generate_data_key(
    KeyId=key_id,
    KeySpec="AES_256"
)

plaintext_key = result["Plaintext"]
encrypted_key = result["CiphertextBlob"]

# Encrypt the sample order locally.
order = b"OrderID: 12345, Item: Pizza"
aesgcm = AESGCM(plaintext_key)
nonce = __import__("os").urandom(12)
ciphertext = aesgcm.encrypt(nonce, order, None)

print("Encrypted order (Base64):",
      base64.b64encode(ciphertext).decode())
print("Encrypted DEK (Base64):",
      base64.b64encode(encrypted_key).decode())

# Decrypt the DEK using AWS KMS.
decrypted_result = kms.decrypt(
    CiphertextBlob=encrypted_key,
    KeyId=key_id
```

```
    )
    decrypted_key = decrypted_result["Plaintext"]

    # Decrypt the order.
    decrypted_order = AESGCM(decrypted_key).decrypt(
        nonce, ciphertext, None
    )

    print("Decrypted order:", decrypted_order.decode())

    # Clear references to key bytes when no longer needed.
    del plaintext_key
    del decrypted_key
```

Expected output format (the encrypted values will differ every time):

```
    Encrypted order (Base64): [encrypted binary data]
    Encrypted DEK (Base64): [KMS-encrypted key data]
    Decrypted order: OrderID: 12345, Item: Pizza
```

Task 5: Sample key rotation policy for an e-commerce application

Key Rotation Policy — E-Commerce Application

Purpose:To protect customer information, order history, payment-related records, and other sensitive data using controlled encryption key management.

1. Rotation frequency
- Enable automatic rotation for eligible AWS KMS customer-managed symmetric encryption keys.
- Review key usage and rotation configuration at least quarterly.
- Rotate application-managed encryption keys at least annually, or sooner if risk, compliance requirements, or the application's design requires it.

2. Notification process

- Send an alert when a key is disabled, scheduled for deletion, or suspected to be compromised.
- Notify the cloud security administrator and application owner.
- Record rotation events and investigate unexpected key-management activity.

3. Handling compromised keys

1. Investigate the suspected compromise and identify affected resources.
2. Restrict or revoke unauthorized key usage and credentials.
3. Create or select a replacement key and re-encrypt affected data where necessary.
4. Verify that the application can securely encrypt and decrypt data using the replacement key.
5. Preserve relevant audit logs and document the incident.
6. Retire the old key only after its remaining dependencies and recovery requirements have been assessed.

4. Review and improvement
Review key permissions, rotation status, access logs, backup requirements, and recovery procedures every quarter. Test the recovery process in a controlled environment.

Changes I would make for my own application: I would separate development and production keys, restrict key administration to authorized security personnel, enable monitoring for suspicious key operations, and test recovery before retiring old keys.